

software as a way of life. I don't see any other reason or way of building things and so if someone asks me, I will always say that they should invest in using and building free software as a career.

I would advise young people in school, college and beyond, to focus more on the sort of things they can do and the depth at which they can learn instead of finding quick or easy ways of doing something. By just consuming what is packaged by others, we also eliminate an opportunity to do things ourselves and hence, learn. If our ability to create something and solve problems is to be based on what we are capable of doing on our own, then choosing free software and hardware to build these solutions is the best way to learn.

Very long ago, I would ask students working with us to follow the 'Linux from Scratch How-to' to manually build their GNU/Linux system without the aid of a distribution. So while it took them a few days to build a usable desktop (as compared to a few minutes that it would take to install Debian on it), the insight and learning they got was unprecedented.

Additionally, I see a lot of young people using free software on proprietary operating systems such as Windows or MacOS. While that might still get the job done, I don't understand how that is the most effective way of building or using free software. There is no limit to what one can do on a GNU/Linux computer. And I would encourage everyone to use GNU/Linux. There is help around if you get stuck – just don't give up when you face a challenge.

Lastly, I would encourage young people to get more involved with hardware experimentation. By understanding how hardware systems work and the role software plays in their functioning, we can build more hardware capable of running free software and can protect our freedom

to compute. Each 'smart device' that we see around us is often easy to build if we choose to do so. And there is still no better way to do that than by using free software and hardware.

Q Any advice for academicians?

To academicians who teach or deal with computing related subjects, I would suggest the following.

Create a culture of learning by doing: To be more effective at teaching a practical subject, one needs to have experience and skills. To gather experience and cultivate these skills, one needs to do things. One cannot do things if one is just a consumer of technology. A very good way to do tasks is to build things yourself.

If someone has dismissed the free software movement as a fad, then they have only viewed it from a short-term or technological perspective.

By building things yourself, you can acquire valuable skills, experience and insights. And these can, in turn, help you become more effective at teaching.

But, how can you build things with 'black boxes'? You can't! To learn to build things, you need tools, directions, knowledge and opportunity. Free software is an ideology that can provide all of these to you. With free software, you have unrestricted access to the resources.

Choose self-hosting: This means that if we need some value from a software or service, instead of deriving that value from proprietary software hosted on the cloud (or only available as a service), we instead, deploy it ourselves on computers that are within our control.

Today, there are very few cases for which you need to use proprietary software (on-premise or on the cloud) in an academic setting. Everything that you need to do can be done very well using GNU/Linux and free software.

Choose free software: When we evaluate our choices of which software, operating system or tool to use for self-hosting the service or solution, we should be focused on choosing free software.

If we chose to self-host using proprietary software, we would have the following constraints:

- We would have no insight into how things work internally (i.e., a wasted opportunity to study and learn because there is no source code provided).
- We would have constraints on how we can use software, for what purpose, with whom, where and in what quantity.
- We would not be able to modify the software to extend it, learn from it or fix it (because there is no source code provided).

So much free software that we use today has originated in academic institutions. Once we start using it, the next step of contribution is more natural and obvious.

Build it yourself: The next step is to do the self-hosting ourselves. Outsourcing the process of self-hosting (even if it is with free software) will not yield the results we want.

Inspire and lead students by example: Lastly, inspire students to learn by putting in similar efforts. By leading by example, we set the benchmark very high. By teaching out of experience, we can help them approach more challenging problems (than what they pursue now).

Q Any advice for the media?

Don't discount and ignore people, projects and businesses because they are 'too idealistic'.